

CHAPTER 05

IMPLEMENTATION

5.1 Mdd prediction system - App.py

```
import os

import joblib

import NumPy as np

import mne

import pickle

from fast Api import Fast API, File, Upload File, HTTP Exception, Form, Request

from fast Api. responses import JSON Response

from typing import List

from SciPy. Stats import skew, kurtosis

from mne. Preprocessing import ICA

from mne. time frequency import psd_array_welch

import shutil

from flask_cors import CORS

from fastapi.middleware.cors import CORSMiddleware

from fastapi.staticfiles import StaticFiles

import matplotlib.pyplot as plt

import pandas as pd

from scipy.fftpack import fft

import pywt

from sklearn.preprocessing import LabelEncoder

# Initialize FastAPI app

app = FastAPI()

# Add CORS middleware

origins = [

    "http://localhost:5173", # Allow React frontend (change if your frontend runs on a different port)

    "http://localhost:5000", # (optional) Add other frontend URLs here if needed

]
```

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Allow all origins (or specify allowed origins)
    allow_credentials=True,
    allow_methods=["*"], # Allow all methods (GET, POST, etc.)
    allow_headers=["*"], # Allow all headers
)
# CORS(app)
app.mount("/static", StaticFiles(directory="static"), name="static")
# Directory for storing uploaded files
UPLOAD_FOLDER = "uploads"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
vhdr_path=""
# Load trained model
# MODEL_PATH = "mw/svm_model.pkl"
MODEL_PATH = "model_weights/xgb_model.pkl"
SCALER_PATH = "scaler/xgb_scaler.pkl"
try:
    model = joblib.load(MODEL_PATH) # Load .pkl model
except Exception as e:
    raise RuntimeError(f"Error loading model: {e}")
scaler = joblib.load(SCALER_PATH) # Only keeping scaler for feature scaling
# Store predictions
predictions = []
plot_paths = []
def preprocess_eeg_data(vhdr_file_path, l_freq=1.0, h_freq=40.0, notch_freq=50):
    """Preprocess EEG data."""
    raw = mne.io.read_raw_brainvision(vhdr_file_path, preload=True)
    eog_channels = ['VPVA', 'VNVB', 'HPHL', 'HNHR']
    raw.set_channel_types({ch: 'eog' for ch in eog_channels if ch in raw.ch_names})
    raw.notch_filter(freqs=[notch_freq], picks='eeg')
    raw.filter(l_freq=l_freq, h_freq=h_freq, picks='eeg')

```

```

raw.set_eeg_reference('average', projection=True)

# ICA for artifact removal
ica = mne.preprocessing.ICA(n_components=20, random_state=97, max_iter=800)
ica.fit(raw)
eog_indices, _ = ica.find_bads_eog(raw)
ica.exclude = eog_indices
raw = ica.apply(raw)

file_path = os.path.splitext(os.path.basename(vhdr_file_path))[0]
print("**file_path",file_path)
plot_path=f"static/{file_path}.png"
print("**plot_path",plot_path)

fig = raw.plot(scalings={"eeg": 75e-6}, n_channels=32, title="EEG Data", show=True)
fig.savefig(plot_path, dpi=300)
plt.close(fig)

return raw

def extract_channel_features(raw, fmin=0.5, fmax=50):
    # Select only EEG channels
    raw.pick_types(eeg=True) # This removes non-EEG channels
    data = raw.get_data()
    channel_names = raw.ch_names
    features = {ch: {} for ch in channel_names}

# Handle categorical data
for col in df.select_dtypes(include=['object']).columns:
    df[col] = LabelEncoder().fit_transform(df[col])

# Handle missing values
df.fillna(df.median(), inplace=True)

return df

def predict_eeg(eo_path, ec_path):
    # Load and Preprocess the Single CSV File
    outut_file = "preprocessed.csv"
    process_and_combine(eo_path, ec_path, outut_file)
    combined_features = pd.read_csv(outut_file)

```

```

X_predict = load_and_preprocess_single_csv(combined_features)
best_model = joblib.load(MODEL_PATH)
# Apply Same Scaling as Training Data
X_predict_scaled = scaler.transform(X_predict)
# Predict on the Single CSV Data
prediction = best_model.predict(X_predict_scaled)
final_prediction = prediction[0]
try:
    probability = best_model.predict_proba(X_predict_scaled)[0] # Probabilities for both classes
    probability_healthy = probability[0] * 100
    probability_mdd = probability[1] * 100
    print(f"Probability of being Healthy: {probability_healthy:.2f}%")
    print(f"Probability of having MDD: {probability_mdd:.2f}%")
except AttributeError:
    print("Prediction probabilities are not available for this model.")
except Exception as e:
    print(f"Error getting prediction probabilities: {e}")
alpha_power = combined_features["alpha_power"].mean()
beta_power = combined_features["beta_power"].mean()
delta_power = combined_features["delta_power"].mean()
theta_power = combined_features["theta_power"].mean()
final_prediction = "MDD" if final_prediction == 1 else "Healthy"
final_prob = probability_mdd if final_prediction == "MDD" else probability_healthy
return final_prediction, final_prob, alpha_power, beta_power, delta_power, theta_power
def visualize_eeg_data(vhdr_path):
    """Visualize EEG data."""
# Convert to Pandas DataFrame
raw = preprocess_eeg_data(vhdr_path)
df = raw.to_data_frame()

# Save as JSON for React
df.to_json("eeg_data.json", orient="records")

```

```

return df

@app.post("/upload/")
async def upload_files(
    vhdrEO: UploadFile = File(...), vmrkEO: UploadFile = File(...), eegEO: UploadFile = File(...),
    vhdrEC: UploadFile = File(...), vmrkEC: UploadFile = File(...), eegEC: UploadFile = File(...)
):
    file_paths = {}
    # Save files to the upload folder
    for file in [vhdrEO, vmrkEO, eegEO, vhdrEC, vmrkEC, eegEC]:
        file_path = os.path.join(UPLOAD_FOLDER, file.filename)
        with open(file_path, "wb") as buffer:
            shutil.copyfileobj(file.file, buffer)
        file_paths[file.filename] = file_path # Store file paths
    print("Files uploaded successfully:", file_paths)
    return {"message": "Files uploaded successfully", "file_paths": file_paths}

```

5.2 Mdd prediction system – model_prediction. ipynb

```

import joblib

import numpy as np

import mne

import numpy as np

from scipy.stats import skew, kurtosis

from mne.preprocessing import ICA

from mne.time_frequency import psd_array_welch

import pandas as pd

from scipy.fftpack import fft

import pywt # For wavelet transform

def preprocess_eeg_data(vhdr_file_path, l_freq=1.0, h_freq=40.0, notch_freq=50):
    raw = mne.io.read_raw_brainvision(vhdr_file_path, preload=True)

    eog_channels = ['VPVA', 'VNVB', 'HPhL', 'HNHR']

    raw.set_channel_types({ch: 'eog' for ch in eog_channels if ch in raw.ch_names})

    raw.notch_filter(freqs=[notch_freq], picks='eeg')

```

```

raw.filter(l_freq=l_freq, h_freq=h_freq, picks='eeg')
raw.set_eeg_reference('average', projection=True)
# ICA for artifact removal
ica = ICA(n_components=20, random_state=97, max_iter=800)
ica.fit(raw)
eog_indices, _ = ica.find_bads_eog(raw)
ica.exclude = eog_indices
raw = ica.apply(raw)
return raw

def extract_channel_features(raw, fmin=0.5, fmax=50):
    # Select only EEG channels
    raw.pick_types(eeg=True) # This removes non-EEG channels
    data = raw.get_data()
    channel_names = raw.ch_names
    features = {ch: {} for ch in channel_names}
    # Time-domain features
    for i, ch in enumerate(channel_names):
        features[ch]['mean'] = np.mean(data[i])
        features[ch]['variance'] = np.var(data[i])
        features[ch]['skewness'] = skew(data[i])
        features[ch]['kurtosis'] = kurtosis(data[i])
        features[ch]['peak_to_peak'] = np.ptp(data[i])
    # Fourier Transform (FFT)
    fft_values = np.abs(fft(data[i]))
    features[ch]['fft_mean'] = np.mean(fft_values)
    features[ch]['fft_std'] = np.std(fft_values)
    features[ch]['fft_max'] = np.max(fft_values)
    # Wavelet Transform (DWT) using Daubechies wavelet (db4) #morle
    coeffs = pywt.wavedec(data[i], 'db4', level=4)
    features[ch]['wavelet_energy'] = sum(np.sum(np.square(c)) for c in coeffs)
    features[ch]['wavelet_entropy'] = 0 # Initialize wavelet_entropy
    for c in coeffs:

```

```

c = c[np.isfinite(c)]
c_norm = c / (np.sum(np.abs(c)) + 1e-10)
features[ch]['wavelet_entropy'] += -np.sum(c_norm * np.log2(c_norm + 1e-10))

# Frequency-domain features using PSD
psd = raw.compute_psd(method='welch', fmin=fmin, fmax=fmax, n_fft=2048)
psd_data = psd.get_data()
freqs = psd.freqs
psd_df = pd.DataFrame(psd_data, columns=freqs, index=channel_names)
bands = {'delta': (0.5, 4), 'theta': (4, 8), 'slow_alpha': (6, 9), 'alpha': (8, 12),
        'beta': (12, 30), 'gamma': (30, 50)}
for band, (low, high) in bands.items():
    band_power = psd_df.loc[:, (freqs >= low) & (freqs <= high)].mean(axis=1)
    for ch in channel_names:
        features[ch][f'{band}_power'] = band_power[ch]

# Frontal Alpha Asymmetry (F3-F4)
if 'F3' in channel_names and 'F4' in channel_names:
    features['F3_F4_alpha_asymmetry'] = features['F4']['alpha_power'] - features['F3']['alpha_power']

# Convert features to DataFrame
features_df = pd.DataFrame(features).T
return features_df

def process_and_combine(eo_file_path, ec_file_path, output_file):
    all_features = []

    # Process EO file
    raw_eo = preprocess_eeg_data(eo_file_path)
    features_eo = extract_channel_features(raw_eo)
    #features_eo['condition'] = 'EO'
    all_features.append(features_eo)

    # Process EC file
    raw_ec = preprocess_eeg_data(ec_file_path)
    features_ec = extract_channel_features(raw_ec)
    #features_ec['condition'] = 'EC'
    all_features.append(features_ec)

```

```
# Combine EO and EC features
combined_features = pd.concat(all_features, keys=['EO', 'EC'], names=['condition', 'channel'])

# Save combined features to a single CSV file
combined_features.to_csv(output_file)

print(f"Features successfully saved to {output_file}")

# return combined_features

import pandas as pd
import joblib
from sklearn.preprocessing import LabelEncoder

# Modified Function to Load and Preprocess a Single CSV File
def load_and_preprocess_single_csv(df):
    Loads and preprocesses a single CSV file for prediction.

    Args:
        file_path (str): The path to the CSV file.

    Returns:
        pandas.DataFrame: Preprocessed DataFrame.

    # Handle categorical data
    for col in df.select_dtypes(include=['object']).columns:
        df[col] = LabelEncoder().fit_transform(df[col])

    # Handle missing values
    df.fillna(df.median(), inplace=True)

    return df

def predict(eo_path, ec_path):
    # Load and Preprocess the Single CSV File
    output_file = "preprocessed.csv"
    process_and_combine(eo_path, ec_path, output_file)
    csv_data = pd.read_csv(output_file)
    X_predict = load_and_preprocess_single_csv(csv_data)

    # Load Pre-trained Model and Scaler
    best_model = joblib.load("../model_weights/xgb_model.pkl")
    scaler = joblib.load("../scaler/xgb_scaler.pkl") # Load the same scaler used during training
```



```

# Apply Same Scaling as Training Data
X_predict_scaled = scaler.transform(X_predict)

# Predict on the Single CSV Data
prediction = best_model.predict(X_predict_scaled)
predicted_class = prediction[0]

if predicted_class == 0:
    prediction_label = "Healthy (0)"
elif predicted_class == 1:
    prediction_label = "MDD (1)"
else:
    prediction_label = "Unknown" # Should not happen in binary classification, but for robustness
print(f"Class: {predicted_class} - {prediction_label}")

try:
    probability = best_model.predict_proba(X_predict_scaled)[0] # Probabilities for both classes
    probability_healthy = probability[0] * 100
    probability_mdd = probability[1] * 100
    print(f"Probability of being Healthy: {probability_healthy:.2f}%")
    print(f"Probability of having MDD: {probability_mdd:.2f}%")
except AttributeError:
    print("Prediction probabilities are not available for this model.")
except Exception as e:
    print(f"Error getting prediction probabilities: {e}")

```

5.3 Mdd prediction system – perprocessing.ipynb

```

import mne
import numpy as np
import matplotlib.pyplot as plt
import os

from mne.preprocessing import ICA

raw = mne.io.read_raw_brainvision('path_to_vhdr_file',preload=True)

raw.plot(show=True)

raw.filter(l_freq=0, h_freq=40) # Bandpass filter

```

```

eog_channels = ['VPVA', 'VNVB', 'HPhL', 'HNHR','Erbs','OrbOcc', 'Mass']
raw.set_channel_types({ch: 'eog' for ch in eog_channels if ch in raw.ch_names})
montage = mne.channels.make_standard_montage('standard_1020')
raw.set_montage(montage,on_missing='ignore')
ica = ICA(n_components=20, random_state=97, max_iter=800)
ica.fit(raw)
sources = ica.get_sources(raw).get_data()
ica.plot_components()
raw.plot(show=True)

```

5.4 Algorithm

Table 5.1 describes the performance of different machine learning models evaluated on the dataset using key classification metrics: accuracy, precision, recall, F1-score, and AUC-ROC. These metrics provide insights into how well each model performs in classifying the data.

TABLE 5.1: Comparison Of Performance Measure

Model	Performance Measure				
	Accuracy (%)	Precision	Recall	F1-score	AUC-ROC
Logistic Regression (LR)	62.14	0.62	0.62	0.61	0.68
Random Forest (RF)	79.17	0.79	0.79	0.79	0.88
Support Vector Machine (SVM)	66.30	0.66	0.66	0.63	0.73
XGBoost	96.00	0.96	0.95	0.95	0.90

Logistic Regression achieved an accuracy of 0.6214, with a precision of 0.6267, recall of 0.6214, and an F1-score of 0.6191. This indicates moderate performance but may struggle with complex decision boundaries.

The XGBoost model outperformed the other models, achieving the highest accuracy of 0.96, along with strong precision (0.96), recall (0.98), and F1-score (0.90). This suggests that XGBoost effectively captures patterns in the data and provides better classification performance.

The Random Forest model showed moderate performance, achieving the accuracy of 0.7917, along with precision (0.7917), recall (0.7917), and F1-score (0.7917).

The Support Vector Machine (SVM) model showed moderate performance, with an accuracy of 0.6630, precision of 0.6695, recall of 0.6630, and an F1-score of 0.6610. While better than Logistic Regression, it does not perform as well as Random Forest.

```

Model: Logistic Regression
Accuracy: 0.6214

```

	precision	recall	f1-score	support
0	0.655914	0.541741	0.593385	563.000000
1	0.596244	0.704251	0.645763	541.000000
accuracy	0.621377	0.621377	0.621377	0.621377
macro avg	0.626079	0.622996	0.619574	1104.000000
weighted avg	0.626674	0.621377	0.619052	1104.000000

```

=====

```

Fig no. 5.1: Performance metrics of Logistic regression

Fig 5.1 illustrates the classification performance of the Logistic Regression model, which achieves an accuracy of 62.14%. The model's precision and recall scores indicate an imbalance between the two classes. Class 0 exhibits higher precision, meaning it correctly identifies non-MDD cases while minimizing false positives. In contrast, Class 1 has better recall, suggesting that the model captures more actual MDD cases but at the cost of increased false positives.

```

=====
Model: Random Forest
Accuracy: 0.7917

```

	precision	recall	f1-score	support
0	0.797853	0.792185	0.795009	563.000000
1	0.785321	0.791128	0.788214	541.000000
accuracy	0.791667	0.791667	0.791667	0.791667
macro avg	0.791587	0.791656	0.791611	1104.000000
weighted avg	0.791712	0.791667	0.791679	1104.000000

```

=====

```

Fig no.5.2: Performance metrics of Random Forest

Fig 5.2 illustrates the classification performance of the Random Forest model, which achieves an accuracy of 79.17%. The precision and recall scores reveal a balanced performance between the two classes. Class 0 demonstrates slightly higher precision, effectively minimizing false positives, while Class 1 maintains a comparable recall, ensuring that actual positive cases are well captured.

```

=====
Model: SVM
Accuracy: 0.6630

```

	precision	recall	f1-score	support
0	0.705376	0.582593	0.638132	563.000000
1	0.632238	0.746765	0.684746	541.000000
accuracy	0.663043	0.663043	0.663043	0.663043
macro avg	0.668807	0.664679	0.661439	1104.000000
weighted avg	0.669536	0.663043	0.660975	1104.000000

```

=====

```

Fig no.5.3: Performance metrics of SVM

Fig 5.3 illustrates the performance of the SVM model, as shown in the figure, achieves an accuracy of 66.30%. The precision score is higher for class 0, indicating that the model effectively minimizes false positives for non-MDD cases. Meanwhile, class 1 exhibits better recall, suggesting that the model captures more actual MDD cases but at the cost of increased false positives.

```

🔥 Validation Accuracy: 0.9550

```

♦ Classification Report:

	precision	recall	f1-score	support
0	0.96	0.94	0.95	2760
1	0.95	0.97	0.96	3600
accuracy			0.96	6360
macro avg	0.96	0.95	0.95	6360
weighted avg	0.96	0.96	0.95	6360

Fig no. 5.4 : Performance metrics of XGBoost

Fig 5.4 illustrates the performance of the XGBoost model, which achieves a high accuracy of 95.50%. The precision score is slightly higher for class 0, indicating effective minimization of false positives for non-MDD cases. On the other hand, class 1 demonstrates superior recall, highlighting the model's ability to accurately identify most actual MDD cases, making XGBoost highly reliable for EEG-based MDD classification with balanced precision and recall across both classes.